Adventist University of Central Africa (AUCA)

Final Project • Technical Report

E-commerce Big Data Analytics

Integrating MongoDB, HBase and Apache Spark

Student	**Mashami Pacifique**
Student ID	101406
Course	Big Data Analytics
Instructor	Temitope Oguntade
Submitted	11 November 2026
Repository	https://github.com/mashami/bigdata-ecommerce

Dataset: 10,000 users • 5,000 products • 75,000 transactions • 300,000 sessions

1. Introduction and system architecture

This project stores each kind of e-commerce data in the system built for it, MongoDB for documents and HBase for time-series events, and uses Apache Spark to clean, join and analyse everything together. The dataset covers 90 days (10 August to 8 November 2026): 10,000 users, 5,000 products in 25 categories, 75,000 transactions and 300,000 browsing sessions with 3.0 million page views.

The pipeline has four stages (Figure 1):

1. **Generate.** The course generator writes JSON files. I converted it from a Colab notebook into a runnable script, added the missing `subcategory_id`, and scaled it to 300,000 sessions (the original targets 2 million) so the full pipeline fits on one laptop.
2. **Store.** Products, users and transactions go to MongoDB. Sessions and daily product metrics go to HBase.
3. **Process.** Spark reads the raw JSON, cleans it into Parquet, and runs the recommendation and SQL analyses.
4. **Integrate.** A Customer Lifetime Value job reads live from MongoDB and HBase, joins them in Spark, and writes each customer's score back into MongoDB.

Everything runs locally: MongoDB 7 and HBase 2.1 in Docker, PySpark 3.5.3 in local mode on a MacBook (Apple Silicon). The generator uses fixed random seeds, so every run reproduces the same data and results.

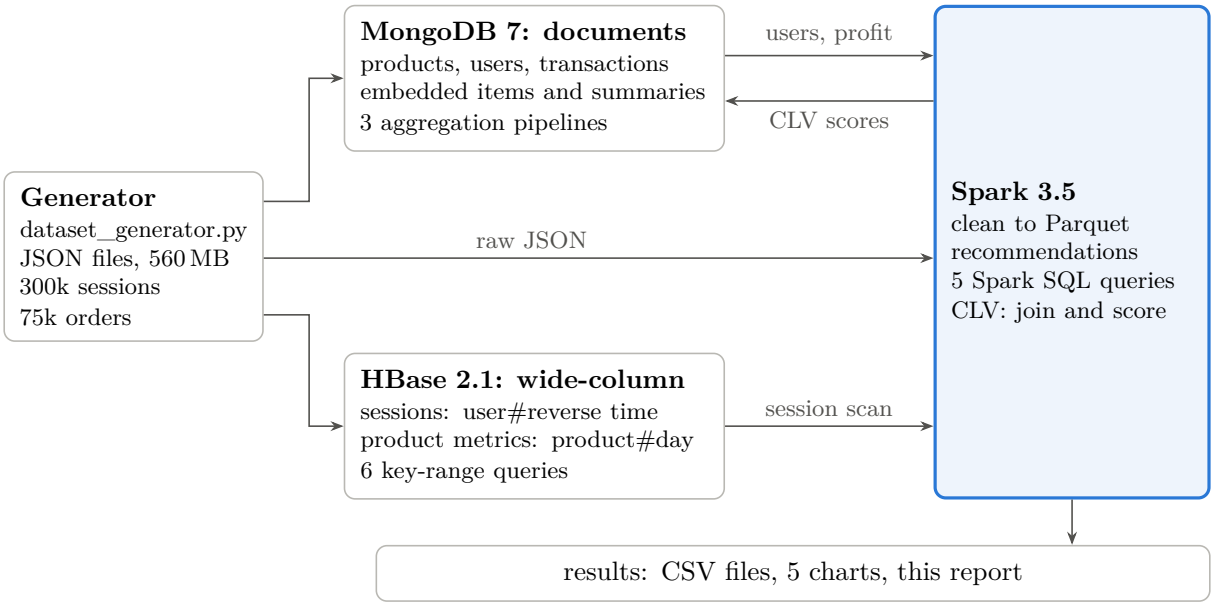


Figure 1: System architecture. Both databases are loaded from the generated JSON, Spark reads all three sources, and the CLV scores flow back into MongoDB.

2. Data modeling

Each schema is designed around the queries that will run against it: MongoDB documents are shaped so one read returns a whole business object, and HBase row keys are shaped so every query is a short range scan.

2.1 MongoDB: document model

Collection	Docs	Design	Why
`products`	5,000	Category and subcategory (name, profit margin) **embedded**; `price_history` embedded as an array	A product page needs its classification in one read. Only 25 categories exist and rarely change, so copying names is cheap.
`transactions`	75,000	Line items **embedded**; each item also carries its `category_id`, `category_name` and `profit_margin` (extended reference pattern)	An order and its items are written and read together. Copied category fields let revenue and profit by category run without a `$lookup`.
`users`	10,000	Profile plus an embedded `purchase_summary`, computed by an aggregation ending in `$merge` (computed pattern)	A profile read shows customer value instantly. Full order lists are **not** embedded: they grow without limit toward the 16 MB document cap.
`categories`	25	The hierarchy, subcategories as an array	Small lookup collection.

Dates are stored as BSON `Date`, not strings, so date ranges and `$dateToString` grouping work. Seven indexes match the query patterns; for example `(user_id, timestamp desc)` serves "this customer's latest orders", and `explain()` confirms an `IXSCAN` instead of a collection scan. The three aggregation pipelines (`mongodb/aggregations.py`) are:

1. **Revenue and profit by category and month**: `$unwind` on items, the order discount spread across its items, monthly buckets with `$dateToString`, profit from the embedded margin.
2. **Top 20 products**: `$group` by product, then `$lookup` into `products` for name, category and remaining stock.
3. **Customer segments**: `$facet` runs two groupings in one pass, by order frequency (`$bucket`) and by spend quartile (`$bucketAuto`). The data has no age field, so segmentation is behavioural.

2.2 HBase: wide-column model

Table	Row key	Column families	Serves
`user_sessions` (300,000 rows)	`user_id#reverse_ts#session_id`	`s`: 16 metadata columns (status, times, device, referrer, cart value). `pv`: one column per page view (`pv:00`, `pv:01`, ...). `cart`: one column per product in the cart.	A user's activity newest first; sessions in a time window; one session's clickstream
`product_metrics` (396,286 rows)	`product_id#YYYYMMDD`	`m`: views, cart adds, cart units, web orders, orders, units sold, revenue	One product over any date range; weekly view-to-purchase conversion

The row key carries the design (example: `user_000042#8206857563000#sess_01802e1738`):

- `user_id` **first** puts all of one user's sessions next to each other, so a user's history is one short scan.
- **The reverse timestamp** (10^{13} minus the start time in ms) makes newer sessions sort first, so "last 10 sessions" reads the first 10 rows of the prefix with no sorting.
- **The session id last** keeps keys unique if two sessions start in the same millisecond.

The three families separate data by how often it is read: listing sessions reads only `s` and never touches page-view data. Page views and cart contents are naturally **sparse**: a session has 4–16 page columns and often an empty cart, and HBase stores nothing for absent columns. Each family keeps one version with a row Bloom filter.

`hbase/query_hbase.py` and `hbase/hbase_shell_commands.txt` implement the queries: a user's 10 latest sessions (prefix scan), sessions in a 30-day window (start and stop rows from the dates), converted sessions only (a `SingleColumnValueFilter` evaluated inside HBase), one session's clickstream (a single `get`), and a product's daily and weekly metrics (range scans). Each returns in milliseconds because it reads only neighbouring rows.

2.3 Why each piece of data lives where it does

Data	System	Deciding factor
Products, users, transactions	MongoDB	Nested objects read whole; ad-hoc analytics need a rich aggregation language and secondary indexes.
Sessions and page views	HBase	Highest volume, append-only, time-ordered, always read by user and time range; scales writes across region servers and stores sparse columns for free.
Daily product metrics	HBase	A fixed (product, date range) access pattern that maps exactly onto a composite row key.

The trade-off is flexibility: HBase answers only the questions its row key was designed for. "All converted sessions from Kenya last week" would need a full scan, so questions like that go to Spark.

3. Spark processing pipeline

Spark turns 560 MB of raw JSON into seven typed Parquet tables in about 100 seconds, then runs the recommendation job and five SQL analyses on them. Every later step reads the Parquet, never the JSON again.

3.1 Cleaning and normalisation (`spark/spark_clean.py`)

1. **Explicit schemas** for every file fix types and avoid a schema-inference pass. `cart_contents`, a JSON object whose keys are product ids, is read as a `MapType`.
2. **Standard formats**: ISO strings parsed to timestamps; categorical text trimmed and lower-cased; country codes upper-cased.
3. **Duplicates and missing values**: primary keys de-duplicated; transactions with no `session_id` kept as channel `offline`; page views without a product (home, search, cart) left null by design.
4. **Validation**: positive quantity and price; line subtotal = quantity × price; order subtotal = sum of lines; total = subtotal − discount. Failing orders, or orders with unknown users or products, are dropped and counted.
5. **Flattening**: items and page views are exploded into their own tables (163,591 and 2,999,732 rows), with category and margin joined onto each item.
6. **Partitioning**: sessions are written partitioned by `session_date`.

Data-quality issue found	Rows	Action
Session marked `converted` but no transaction exists (the generator stops creating orders once its target is reached)	23,071	Relabelled `converted_no_order`
User `last_active` earlier than their real last session or order	9,856	Recomputed from activity
Transaction with no session	37,230	Kept, channel `offline`

All value, key and timestamp checks passed with zero failures, as expected of synthetic data.

3.2 Product recommendations (`spark/spark_recommendations.py`)

The job builds association rules $X \to Y$, where N is the number of baskets:

$$\text{support} = \frac{n(X \cap Y)}{N}, \qquad \text{confidence} = \frac{n(X \cap Y)}{n(X)}, \qquad \text{lift} = \frac{\text{confidence}}{n(Y)/N}.$$

- **"Users who bought X also bought Y"**: a basket is everything one user bought (single orders hold only 1–3 items). A self-join with $x < y$ counts each pair once; pairs shared by at least 2 users are kept: 88,464 pairs.
- **"Viewed together"**: the same method on products viewed in each session, kept at 3 or more shared sessions: 7,135 pairs.
- **Top 5 per product**: shared purchases score 3, shared views 1, ranked with `row_number()` over a window partitioned by product. 4,761 of 5,000 products receive recommendations.

A **random-shuffle baseline** tests whether these patterns are real: it keeps every user's basket size but assigns products at random, then counts pairs again. The shuffled data produced **88,215** repeated pairs against **88,464** in the real data (ratio 1.00), and lift between the 25 categories stayed between 1.02 and 1.08. The recommender works, but this dataset holds no genuine affinity because the generator chooses products at random. High lifts on individual pairs (up to 40) are an artefact of small counts, which is why a baseline matters before acting on such rules.

3.3 Spark SQL analytics (`spark/spark_sql.py`)

The Parquet tables are registered as views named after the system where each dataset lives (`mongo_users`, `mongo_transactions`, `mongo_items`, `mongo_products`, `hbase_sessions`, `hbase_page_views`), so each query reads as one SQL statement across both databases, as it would with the MongoDB and HBase Spark connectors.

	Question	SQL techniques
Q1	Where do sessions drop out of the funnel, by device and referrer?	CTE, conditional aggregation, `GROUP BY ROLLUP`
Q2	How does daily revenue move?	`AVG() OVER (ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)`, running total
Q3	Which categories lead each month, and are they growing?	`RANK()`, `LAG()`, revenue per day for partial months
Q4	Are the most viewed products the best sellers?	HBase views `JOIN` MongoDB sales, `RANK()` on both
Q5	Do customers who browse more spend more?	MongoDB users `JOIN` HBase sessions, `NTILE(4)`, `CORR()`

3.4 Optimisation choices

- **Parquet** (columnar, compressed) instead of re-reading JSON: later jobs read only the columns they need.
- **Date partitioning**: the physical plan for a one-week query shows `PartitionFilters` on `session_date`, so Spark reads 7 of 90 folders.
- **Explicit schemas** remove a full inference pass over the input.
- **16 shuffle partitions** instead of the default 200 on one machine; `cache()` on basket tables reused by several self-joins; 4 GB driver memory for the multi-line JSON files.

4. Integrated analytics: Customer Lifetime Value

The integrated query predicts each customer's 12-month value from three systems at once. It finds that the top 25% of customers hold 42% of future value, and flags 1,103 lower-value customers who browse heavily but abandon carts.

Business question. Which customers will be worth the most over the next 12 months, how does their behaviour differ, and which lower-value customers are worth a retargeting campaign?

Data sources. Profiles and purchase history come from MongoDB; browsing engagement (sessions, time on site, abandoned carts) comes from HBase; Spark joins and models them. No single system holds all three.

4.1 Workflow (`integration/clv_integration.py`)

1. **MongoDB, pushed-down aggregation.** The script reads each user's `purchase_summary` and runs a pipeline *inside* MongoDB that computes per-user estimated profit from the margin embedded in each line item. Only 10,000 summary rows cross the network, not 75,000 orders.
2. **HBase, narrow scan.** A full scan of `user_sessions` requests only five columns of family `s`; the user id is read from the row key itself. 300,000 rows stream in batches of 5,000.
3. **Spark, join and model.** Sessions are aggregated per user (count, minutes, abandoned carts and their value), joined with both MongoDB extracts, and scored.
4. **MongoDB, write-back.** Each user document receives a `clv` sub-document (score, tier, retention probability, retarget flag) through one unordered `bulk_write`, plus an index on `clv.tier`. Spark's result becomes operational data that a normal profile read returns.

If HBase is unavailable, `-sessions-from parquet` reads the same sessions from Parquet, so the analysis degrades gracefully instead of failing.

4.2 The CLV model

The model is deliberately simple and explainable:

$$\mathrm{CLV}_{12m} \;=\; \frac{\text{estimated profit over 90 days}}{3} \times 12 \times p_{\text{retain}}$$

Estimated profit is net revenue after discounts multiplied by each item's subcategory margin. The retention probability comes from recency: 0.9 if the last order was within 14 days, 0.7 within 30, 0.4 within 60 and 0.15 otherwise. Customers are split into CLV quartiles: Platinum, Gold, Silver and Bronze.

4.3 Results and performance

- Average 12-month CLV runs from **$11,345** (Platinum) to **$2,631** (Bronze), a 4.3× gap; the top tier holds 42.1% of total predicted value.
- Value is driven by **recency** (correlation with CLV −0.49) and **frequency** (10.4 orders for Platinum against 4.7 for Bronze). Browsing time matters far less (correlation +0.15).
- **Retargeting list:** 1,103 Silver and Bronze customers are in the top 25% for browsing time and abandoned 3 or more carts, leaving **$14.5 million** in abandoned cart value.

Each system does the work it is fastest at: MongoDB aggregates next to the data, HBase serves a narrow sequential scan, and Spark performs the many-to-many join that neither database does well. At production scale the Python Thrift scan would become the bottleneck; the next step is the Spark HBase connector, which reads regions in parallel, and the MongoDB Spark connector with the same pipeline pushed down.

5. Key findings and visualizations

The five findings below point to the two actions with the most value: fix the checkout step, and restock best sellers before they sell out. All charts are produced by `visualization/make_charts.py`.

Sales over time. Revenue holds steady at about $884,000 per full day, $79.6 million over 90 days, with no trend, weekly cycle or seasonal peak (Figure 2). The low first and last points are partial days. A flat

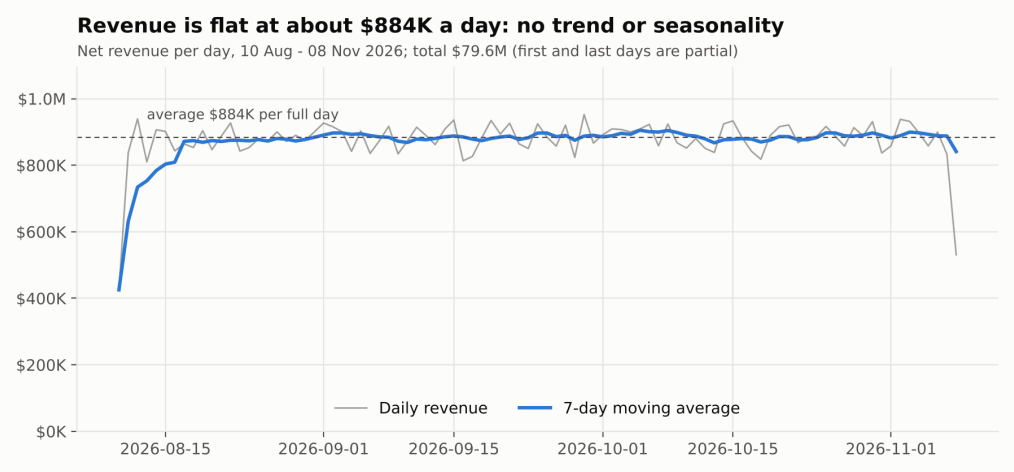


Figure 2: Daily net revenue and 7-day moving average (Spark SQL Q2).

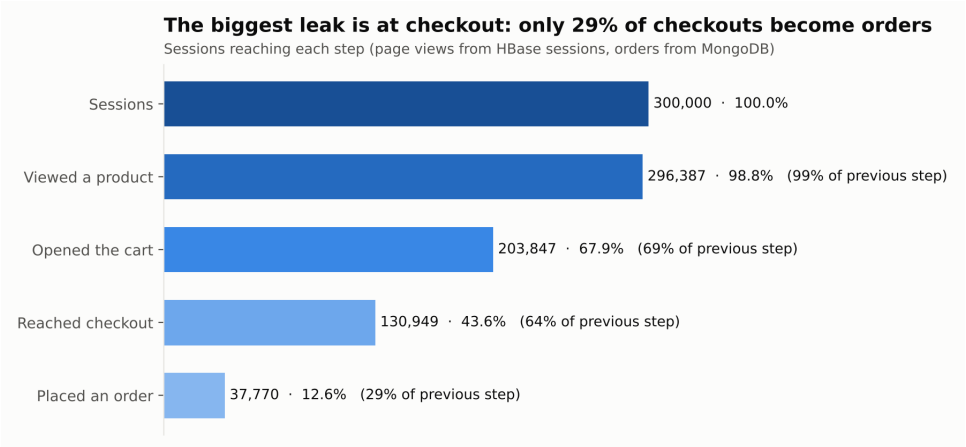


Figure 3: Conversion funnel: page views from HBase sessions, orders from MongoDB (Spark SQL Q1).

line is expected from a generator that spreads orders uniformly; on real data the same query would expose promotions and seasonality.

Conversion funnel. Almost every session views a product and two thirds open the cart, but only 29% of sessions that reach checkout end in an order (Figure 3). Part of this drop is the generator artefact in Section 3.1, but even counting every session labelled "converted", over half of checkouts are lost. **Action:** simplify checkout and target the abandoned carts identified in Section 4.

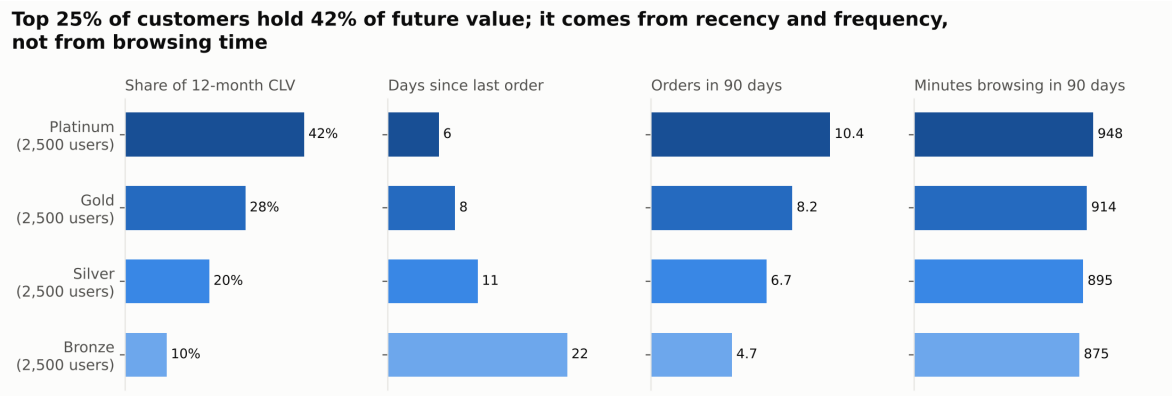


Figure 4: Customer segmentation by 12-month CLV tier (integrated MongoDB + HBase + Spark query).

Customer segmentation. Customer value is concentrated, and it comes from buying recently and often, not from browsing longer (Figure 4). Bronze customers browse almost as much as Platinum ones but last ordered 22 days ago against 6, and placed half as many orders. **Action:** spend retention budget on re-engaging customers before they lapse rather than on increasing time on site.

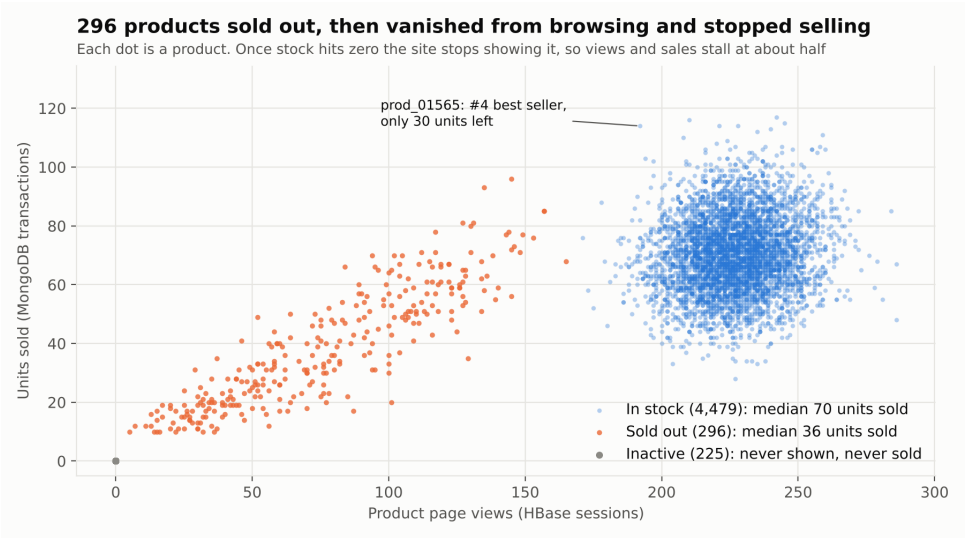


Figure 5: Product page views (HBase) against units sold (MongoDB), by stock status (Spark SQL Q4).

Product performance. Joining HBase views with MongoDB sales revealed a pattern neither database showed alone: 296 products sold out during the window, the site then stopped showing them, and they sold about half as many units as products that stayed in stock (Figure 5). A further 225 inactive products were never shown or sold, and the fourth-best-selling product, `prod_01565`, has only 30 units left. **Action:** a reorder alert driven by the sales rate in HBase `product_metrics`, triggered before stock reaches zero.

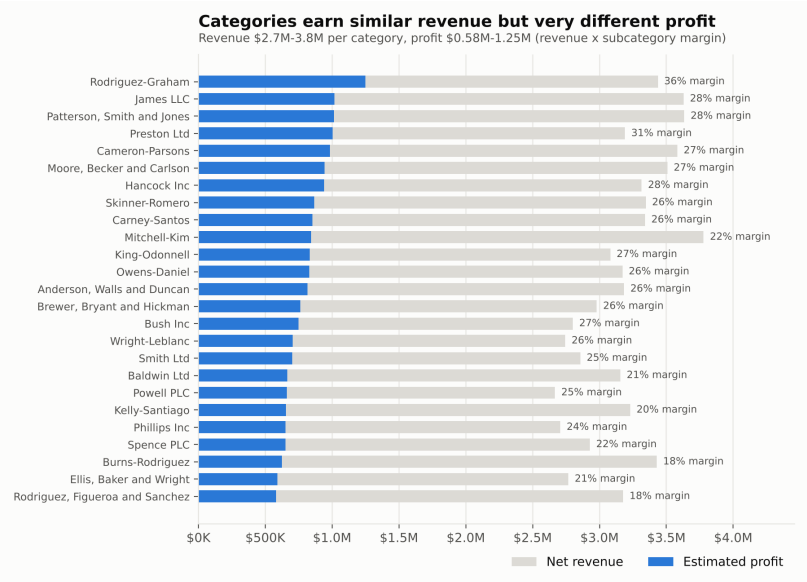


Figure 6: Net revenue and estimated profit per category (Spark SQL Q3, MongoDB aggregation 1).

Revenue versus profit. All 25 categories earn similar revenue, but estimated profit ranges about twofold because subcategory margins run from 18% to 36% (Figure 6). Mitchell-Kim earns the most revenue but ranks 10th on profit (22% margin), while Rodriguez-Graham leads on profit from 6th place on revenue (36% margin). **Action:** judge categories by profit, and favour high-margin categories in recommendations and promotions.

6. Scalability, technology choices, limitations and future work

6.1 Scalability

The design scales out without code changes; only the deployment changes.

- **MongoDB** would be sharded on a hashed `user_id` for `users` and `transactions`, keeping one customer's

orders on one shard; aggregation pipelines already run per shard and merge.

- **HBase** splits rows across region servers by key range. Sequential user ids would concentrate new writes on one region, so production tables would be pre-split at user boundaries (`SPLITS`, shown in the shell file) or use a short hash "salt" prefix, at the cost of fanning a user's scan out to each salt bucket.
- **Spark** code uses DataFrames with no driver-side loops over data, so the same jobs run on a cluster. The parts that would not scale as written are the Python Thrift scan of HBase and the pandas hand-off from MongoDB; both would be replaced by the official Spark connectors, and loading would use HBase bulk loads and parallel MongoDB writers.

6.2 Technology selection

Task	Chosen	Why not the others
Catalogue, profiles, orders	MongoDB	HBase cannot run ad-hoc aggregations or secondary-index lookups; Spark is not a serving store.
Session history by user and time	HBase	At billions of append-only events, HBase's sorted, region-split storage and sparse columns are cheaper and faster for key-range reads than MongoDB indexes.
Daily product time series	HBase	A fixed (product, date) access pattern maps exactly onto a composite row key.
Cleaning, recommendations, joins, CLV	Spark	Neither database joins large datasets well; Spark parallelises joins, window functions and full scans.

6.3 Limitations

- **Synthetic data has no real patterns.** The shuffle test shows product affinities are chance, and device, referrer and country show no differences in conversion. The findings describe the generator's rules more than real customer behaviour.
- **Generator artefacts**: 23,071 "converted" sessions without orders, an inflated drop at the order step, and `last_active` dates that contradict activity. These were corrected or flagged, but they affect funnel and recency figures.
- **Reduced scale, single node**: 300,000 sessions instead of 2 million so a laptop could hold the data (ratios between entities are preserved), and HBase runs standalone under emulation, so timings do not represent a cluster.
- **Simple CLV model**: fixed retention steps, a 90-day window and no discounting. It ranks customers well but is not a calibrated forecast.
- **No demographics**: no age or gender, so segmentation is behavioural only.

6.4 Future work

- Replace the CLV rules with a probabilistic model (BG/NBD for purchase frequency with Gamma-Gamma for order value), validated on a held-out period.
- Add a streaming path: Spark Structured Streaming writing session events to HBase and incrementing `product_metrics` counters in real time.
- Re-run the recommendation baseline on real transactions, where a ratio well above 1 would confirm genuine affinities.

Appendix: sample queries

The complete code is in the repository; these excerpts show one query from each system.

A. MongoDB aggregation: revenue and profit by category and month

```
db.transactions.aggregate([
  { $match: { status: { $in: ["completed", "shipped", "delivered"] } } },
  { $addFields: { paid_ratio: { $divide: ["$total", "$subtotal"] } } },
  { $unwind: "$items" },
  { $group: {
      _id: { month: { $dateToString: { format: "%Y-%m", date: "$timestamp" } },
             category: "$items.category_name" },
     net_revenue: { $sum: { $multiply: ["$items.subtotal", "$paid_ratio"] } },
     est_profit: { $sum: { $multiply: ["$items.subtotal", "$paid_ratio",
                                       "$items.profit_margin"] } } } },
  { $sort: { "_id.month": 1, net_revenue: -1 } }
])
```

B. HBase shell: schema and session queries

```
create 'user_sessions', {NAME => 's', VERSIONS => 1, BLOOMFILTER => 'ROW'},
                        {NAME => 'pv', VERSIONS => 1}, {NAME => 'cart', VERSIONS => 1}
create 'product_metrics', {NAME => 'm', VERSIONS => 1, BLOOMFILTER => 'ROW'}

# 5 most recent sessions of one user (reverse timestamp = newest first)
scan 'user_sessions', {ROWPREFIXFILTER => 'user_000042#', COLUMNS => ['s'], LIMIT => 5}

# only converted sessions, filtered inside HBase
scan 'user_sessions', {ROWPREFIXFILTER => 'user_000042#', COLUMNS => ['s'],
  FILTER => "SingleColumnValueFilter('s','status',=,'binary:converted',true,true)"}

# one product's daily metrics for September
scan 'product_metrics', {STARTROW => 'prod_02076#20260901', STOPROW => 'prod_02076#20261001'}
```

C. Spark SQL: conversion funnel across HBase and MongoDB data (Q1)

```
WITH session_steps AS (
  SELECT s.session_id, s.device_type, s.referrer,
         MAX(CASE WHEN p.page_type = 'product_detail' THEN 1 ELSE 0 END) AS viewed_product,
         MAX(CASE WHEN p.page_type = 'cart' THEN 1 ELSE 0 END) AS visited_cart,
         MAX(CASE WHEN p.page_type = 'checkout' THEN 1 ELSE 0 END) AS reached_checkout,
         MAX(CASE WHEN s.has_order THEN 1 ELSE 0 END) AS placed_order
  FROM hbase_sessions s JOIN hbase_page_views p ON p.session_id = s.session_id
  GROUP BY s.session_id, s.device_type, s.referrer)
SELECT COALESCE(device_type, 'ALL') AS device_type, COALESCE(referrer, 'ALL') AS referrer,
       COUNT(*) AS sessions, SUM(viewed_product) AS viewed_product,
       SUM(visited_cart) AS visited_cart, SUM(reached_checkout) AS reached_checkout,
       SUM(placed_order) AS placed_order,
       ROUND(100 * SUM(placed_order) / COUNT(*), 2) AS conversion_pct
FROM session_steps
GROUP BY ROLLUP(device_type, referrer)
```